



Habib University

CE 321/CS 330 Computer Architecture – Fall 2025

Instructor: Dr. Shehzad Hasan

Time = 50 minutes

Quiz 04

Max Points: 20

Name: _____

ID: _____

Instructions:

- i. **Smart watches, laptops, and similar electronics are strictly NOT allowed.**
- ii. **Answer sheets should contain all steps, working, explanations, and assumptions.**
- iii. Attempt the quiz on clean papers with black/blue ink.
- iv. Print your name and HU ID on all sheets.
- v. Turn in your question paper along with your answer sheets.
- vi. You are not allowed to ask/share your method or answer with your peers. The work submitted by you is solely your own work. Any violation of this will be the violation of HU Honor code and proper action will be taken as per university policy if found to be involved in such an activity.

CLO Assessment:

This assignment assesses students for the following course learning outcomes.

Course Learning Outcomes		CLO Assessed
CLO 1	<i>Explain</i> the role of ISA in modern processors and instruction encodings and assembly language programming	
CLO 2	<i>Explain</i> the architecture and working of a single cycle processor	
CLO 3	<i>Design</i> the architecture to mitigate issues of a pipelined processor	
CLO 4	<i>Analyze the</i> performance of cache operations	✓

Question 1 [10 points]:

- a) Assume we have 64-bytes SRAM. Let's say we are implementing a 2 - **half word** per block direct mapped cache using this memory. Organize this memory in form of a direct mapped cache. Fill the table for the given cache indexes, first three block numbers that could be stored in that cache index and halfword addresses for each block. Do this for all cache lines, first one is done for you. **[5 points]**

How to get cache index?

Because the cache is 64 bytes and the line size is 4 bytes (2 halfwords) it has $64/4 = 16$ lines

How to get block numbers?

Because there are 16 lines then after 16 blocks (B0 to B15) the next will go to line 0, so all blocks divisible by 16 are going to go to cache line 0 and so on.

How to get halfword addresses?

Because the line size is 4 bytes and halfword size is 2 bytes each MM block / cache line can contain 2 halfwords. So, i^{th} MM Block will have half-word addresses starting from $4 \times \text{MM block number}$ and then there will be 2 halfwords in order.

Cache index	Block number	Half Word addresses
0	B0, B16, B32, ...	[0, 2], [64, 66], [128, 130], ...
1	B1, B17, B33, ...	[4, 6], [68, 70], [132, 134], ...
2	B2, B18, B34, ...	[8, 10], [72, 74], [136, 138], ...
14	B14, B30, B46, ...	[56, 58], [120, 122], [184, 186], ...
15	B15, B31, B47, ...	[60, 62], [124, 126], [188, 190], ...

- b) Assume a 16KiB direct mapped cache having 8-words per block. Calculate number of bits needed for tag, cache index and block offset, if the address bus width is 28 bits. Also, find the actual size of cache and calculate the ratio of actual size compared to the data size. **[5 points]**

Each word is 4 bytes.

Line size = $8 \times 4 = 32$ bytes

lines = $16 \text{ K} / 32 = 512$ lines

Offset = $\log_2(32) = 5$ bits

Block = $\log_2(512) = 9$ bits

Tag = $28 - 9 - 5 = 14$ bits

Actual Size:

Each cache line has tag bits, plus one valid bit.

So total size = # cache lines $\times$ (valid bit + tag bits + data bits)
= $512 \times (1 + 14 + 32 \times 8)$
= 138,752 bits

Data size:

$512 \times 32 \times 8 = 131072$ bits

Ratio = $138752/131072 = 1.0586$

Question 2 [10 points]:

- a) Let's say we have a cache with 64 blocks. The cache is organized as a direct-mapped cache, having 4-words per block. For the given word addresses, fill the table: 64, 128, 464, 2044, 1280, 4080, 1016, 2520, 1500, 260, 68, 132. **[5 marks]**

(Note: Each word contains 4 bytes)

Word addr	MM Block #	Cache index	Hit/ Miss	Miss type	Word addr	MM Block #	Cache index	Hit/ Miss	Miss type
64	4	4	M	Comp	1016	63	63	M	Comp
128	8	8	M	Comp	2520	157	29	M	Comp
464	29	29	M	Comp	1500	93	29	M	Comp
2044	127	63	M	Comp	260	16	16	M	Comp
1280	80	16	M	Comp	68	4	4	H	-
4080	255	63	M	Comp	132	8	8	H	-

How to get MM Block #?

Each word is of 4 bytes, so the first thing would be to determine which word number is it. E.g. 64 is the $64/4 = 16^{\text{th}}$ word. The second thing would be to determine the block number by dividing this with block size (in words) and taking the floor of it. $16/4 = 4.0$ so 4.

You can also do it by dividing this address directly with the block size in bytes. i.e. $64/16 = 4.0$ so again 4.

How to get the cache index?

Find the number of cache lines first. There are 64 lines (blocks) so each MM block number has to be divided by 64 and we are looking for the remainder. i.e. MM block no. % (# cache lines)
So $4 \% 64 = 4$.

Miss type?

If it's the first reference to the block it's a compulsory miss. If this block was referenced before and we removed it because the cache set cannot keep it, then next reference to it (because it's a miss) would be a conflict miss. If the working set of the program is more than what a fully associative cache could hold then it's a capacity miss.

- b) Now assume that we change the original cache to a 4-way set-associative cache with 64 sets/cache indexes. This cache is also organized as 4-words per block. For the given addresses, fill the table: 64, 128, 464, 2044, 1280, 4080, 1016, 2520, 1500, 260, 68, 132 [5 marks]

Word addr	Block addr	Set index	Hit/ Miss	Miss type	Word addr	Block addr	Cache index	Hit/ Miss	Miss type
64	4	4 (way 0)	M	Comp	1016	63	63 (way 2)	M	Comp
128	8	8 (way 0)	M	Comp	2520	157	29 (way 1)	M	Comp
464	29	29 (way 0)	M	Comp	1500	93	29 (way 2)	M	Comp
2044	127	63 (way 0)	M	Comp	260	16	16 (way 1)	M	Comp
1280	80	16 (way 0)	M	Comp	68	4	4 (way 0)	H	-
4080	255	63 (way 1)	M	Comp	132	8	8 (way 0)	H	-

How to get the set index?

Find the number of cache sets. There are 64 sets as mentioned in the question so each MM block number has to be divided by 64 and we are looking for the remainder. i.e. MM block no. % (# of sets)
So $10 \% 64 = 10$.

How to determine way?

If it's a first block to the set assign it way 0, otherwise the next available way. If the block is already there it's a hit.